LABORATORY

LAB 03

Fundamentals of Serial Monitor

Monitoring System Status with EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1 Introduction . . . 2
1.1 Lab Overview . . . 2
1.2 Objectives . . . 2
2 Background . . . 2
2.1 Serial Communication and Serial Monitor . . . 2
2.2 UART and Asynchronous Communication . . . 2
2.3 Baud Rate . . . 3
2.4 Basic UART Frame Structure . . . 3
3 Hardware Setup . . . 3
3.1 Required Hardware . . . 3
3.2 Interfaces Used . . . 4
4 EduFramework API . . . 4
4.1 `Serial1_begin()` . . . 4
4.2 `Serial1_print()` . . . 5
4.3 `Serial1_println()` . . . 5
5 Lab Exercise . . . 6
5.1 Requirements . . . 6
5.2 Program . . . 6
5.3 Verification . . . 7
6 Extension . . . 8
6.1 Printing Integer Values . . . 8
6.2 Printing Floating-Point Values . . . 8
6.3 Extension Exercise: Counting Button Presses . . . 9
6.4 `Serial1` and `Serial2` . . . 9
7 References . . . 9

1 Introduction

1.1 Lab Overview

During embedded system development, observing the internal state of a program is necessary for verifying software behavior and monitoring runtime data. A common method is to transmit information from the microcontroller to a computer through serial communication and display the data on a Serial Monitor.

UART (Universal Asynchronous Receiver/Transmitter) is an asynchronous serial communication mechanism commonly integrated into microcontrollers. Data is transmitted sequentially, one bit at a time, between devices without requiring a shared clock line for transmission and reception [1] .

This lab is designed to introduce the Serial Monitor through the `Serial1` interface of EduFramework. The state of the onboard push button is transmitted to the computer for direct observation on the Serial Monitor. The Serial Monitor therefore becomes a fundamental tool that can be reused in subsequent labs to observe sensor values, system states, and processing results.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Explain the basic principles of asynchronous serial communication and the role of UART in data transmission.
2. Describe the roles of TX, RX, baud rate, and the basic structure of a UART frame.
3. Use the `Serial1_begin()` , `Serial1_print()` , and `Serial1_println()` APIs to transmit data to the Serial Monitor.
4. Build and verify an application that monitors push-button events on the Serial Monitor.

2 Background

2.1 Serial Communication and Serial Monitor

In serial communication, data is transmitted sequentially one bit at a time rather than transmitting multiple bits simultaneously over multiple signal lines. This method is widely used in embedded systems to exchange data between microcontrollers and other devices [1] .

A Serial Monitor is a tool for displaying data transmitted through a serial interface. In this lab, the Serial Monitor is used as an observation channel from the MaaZEDU Development Board to the computer. The program can transmit text strings that describe states or events to support verification of system operation.

2.2 UART and Asynchronous Communication

UART performs asynchronous serial communication. Unlike synchronous communication, the two endpoints do not use a shared clock line to determine the timing of data transmission and reception. Instead, the devices must use compatible communication parameters, particularly the transmission rate [2] .

A typical UART interface uses two signal directions:

Signal	Name	Function
TX	Transmit	Transmits data from the current device to the receiving device.
RX	Receive	Receives data transmitted by another device.

Table 3.2.1. Basic UART signal directions

This lab uses only the data transmission direction from the S32K144 to the computer. Therefore, the main content focuses on transmitting data through `Serial1`; data reception will be introduced in a later lab.

2.3 Baud Rate

Baud rate represents the symbol rate of the communication interface. In typical binary UART communication, each symbol corresponds to one bit; therefore, the baud rate determines the transmission interval of each bit. Both endpoints must use compatible configurations so that the transmitted data can be interpreted correctly [1].

For example, when the program initializes:

```
Serial1_begin(9600U);
```

the `Serial1` interface is configured to operate at `9600` baud. The Serial Monitor on the computer must also use the same baud rate.

Note

With the current default clock configuration of EduFramework, Serial communication should use low baud rates to ensure stable operation. In this laboratory series, `9600 baud` is recommended and used as the default configuration for the Serial Monitor.
This limitation is related to the current framework configuration and is not a general limitation of the LPUART peripheral on the S32K144.

2.4 Basic UART Frame Structure

Because UART does not use a shared clock line, data must be organized into a frame so that the receiver can identify the beginning and end of the data. A basic UART frame consists of a start bit, data bits, and a stop bit; a parity bit may also be included depending on the configuration [2].

A common configuration is `8N1`, consisting of:

Component	Meaning
1 Start bit	Marks the beginning of a data frame.
8 Data bits	Contains the eight data bits to be transmitted.
No parity	No parity bit is used.
1 Stop bit	Marks the end of the frame before the next data is transmitted.

Table 3.2.2. General structure of an 8N1 UART configuration

3 Hardware Setup

3.1 Required Hardware

This lab uses the onboard push button on the MaaZEDU Development Board and the PlatformIO Serial Monitor; therefore, no additional components or external circuit are required.

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
USB Cable	Connects the board to the computer for power, program upload, and use of the Serial Monitor.

Table 3.3.1. Hardware used in the lab exercise

3.2 Interfaces Used

The `BTN0` push button is used to generate the event to be monitored. EduFramework maps `BTN0` to the `PTC12` pin of the S32K144 [5] .

Component	Logical Pin	MCU Pin	Function
Onboard push button	`BTN0`	`PTC12`	Digital Input

Table 3.3.2. Digital Input mapping used in the lab exercise

For Serial data, EduFramework uses `Serial1` for the Serial Monitor and debug information through the board debug interface. `Serial1` is mapped to the `LPUART1` peripheral [4] .

Interface	Peripheral	Role in this lab
`Serial1`	`LPUART1`	Transmits data from the S32K144 to the Serial Monitor on the computer.

Table 3.3.3. Serial interface used in the lab exercise

The data flow in this lab can be represented as follows:
`BTN0` → program running on the S32K144 → `Serial1` → Serial Monitor.

4 EduFramework API

This lab uses three basic `Serial1` APIs: `Serial1_begin()` to initialize the interface, `Serial1_print()` to transmit a string, and `Serial1_println()` to transmit a string followed by a line termination. These APIs belong to the Arduino-style API layer of EduFramework [4] .

4.1 `Serial1_begin()`

`Serial1_begin()` initializes the `Serial1` interface with the specified baud rate.

`Serial1_begin`

Initializes `Serial1` for data transmission and reception at the specified baud rate.

Syntax

```
Serial1_begin(baudRate);
```

Parameters

Parameter	Type / accepted value	Description
`baudRate`	Baud rate	Transmission rate used for Serial communication.

Return value

No return value.

Example:

```
Serial1_begin(9600U);
```

In this laboratory series, `9600 baud` is used for the Serial Monitor with the current EduFramework configuration.

4.2 `Serial1_print()`

`Serial1_print()` transmits a string through `Serial1` without automatically terminating the line.

`Serial1_print`

Transmits the specified string through `Serial1` .

Syntax

```
Serial1_print(text);
```

Parameters

Parameter	Type / accepted value	Description
`text`	String	Content to be transmitted to the Serial Monitor.

Return value

No return value.

Example:

```
Serial1_print("System status: ");
```

Subsequent transmitted data continues to appear on the same line until a line termination operation is performed.

4.3 `Serial1_println()`

`Serial1_println()` transmits a string through `Serial1` and automatically terminates the line after the transmitted string [4] .

`Serial1_println`

Transmits the specified string through `Serial1` and terminates the line.

Syntax

```
Serial1_println(text);
```

Parameters

Parameter	Type / accepted value	Description
`text`	String	Content to be transmitted to the Serial Monitor.

Return value

No return value.

Example:

```
Serial1_println("System started.");
```

Unlike `Serial1_print()`, data transmitted using `Serial1_println()` terminates the current line, and subsequent content is displayed on a new line. The corresponding organization of `print()` and `println()` is also used in the Arduino Serial model [3].

5 Lab Exercise

5.1 Requirements

Build a program that uses `BTN0` as an event source and displays the push-button state on the Serial Monitor.

The program must satisfy the following requirements:

- Initialize `Serial1` at `9600` baud.
- Display a message when the system starts.
- When `BTN0` is pressed, the Serial Monitor displays the corresponding message exactly once.
- When `BTN0` is released, the Serial Monitor displays the corresponding message exactly once.
- Holding the button does not cause the same message to be printed continuously.

5.2 Program

In `src/main.c`, implement the program as follows:

```
#include "Arduino.h"

int main(void)
{
    bool pressed = false;

    setup();

    pinMode(BTN0, INPUT);

    Serial1_begin(9600U);

    Serial1_print("EduFramework ");
    Serial1_println("Serial Monitor");
    Serial1_println("System started.");

    while (1)
    {
        if ((HIGH == digitalRead(BTN0)) && (false == pressed))
        {
            Serial1_println("BTN0 pressed.");
            pressed = true;
        }

        if ((LOW == digitalRead(BTN0)) && (true == pressed))
        {
            Serial1_println("BTN0 released.");
            pressed = false;
        }
    }

    return 0;
}
```

Listing 3.5.1. Program for monitoring push-button events using the Serial Monitor

After `setup()` initializes EduFramework, `BTN0` is configured as a Digital Input, and `Serial1_begin(9600U)` initializes Serial communication at `9600` baud.

The `Serial1_print()` and `Serial1_println()` APIs are used to display startup information. In the main loop, the `pressed` variable records the processing state of the push button. This mechanism follows the event-detection principle used in Lab 02; however, instead of controlling a Digital Output, the event is converted into information that can be observed on the Serial Monitor.

When `BTN0` changes to the pressed state, the `BTN0 pressed.` message is transmitted once. When the button is released, the program transmits `BTN0 released.` and becomes ready to process the next press event.

5.3 Verification

Build and upload the program to the MaaZEDU Development Board. Then open the Serial Monitor and configure the baud rate to `9600`.

Observe the displayed content when the program starts, then press, hold, and release `BTN0` in sequence.

EXPECTED RESULT

The Serial Monitor displays the system startup information. Each time `BTN0` is pressed, the `BTN0 pressed.` message appears exactly once; when the button is released, the `BTN0 released.` message appears exactly once. Holding the button does not cause the same message to repeat continuously.

6 Extension

The basic APIs in the main exercise focus on transmitting strings. In measurement and monitoring applications, the observed data often also includes numeric values such as event counts, ADC values, or calculated sensor results. EduFramework provides additional APIs for directly printing integer and floating-point values through `Serial1` [4] .

6.1 Printing Integer Values

`Serial1_printInt()` transmits a signed integer value through `Serial1`, while `Serial1_printlnInt()` performs the same operation and terminates the line after the value.

API	Syntax	Function
`Serial1_printInt()`	`Serial1_printInt(value);`	Prints an integer value without automatically terminating the line.
`Serial1_printlnInt()`	`Serial1_printlnInt(value);`	Prints an integer value and terminates the line.

Table 3.6.1. APIs for printing integer values through Serial1

Example:

```
Serial1_print("Value: ");
Serial1_printlnInt(value);
```

Separating the descriptive text from the numeric value makes the information on the Serial Monitor easier to read.

6.2 Printing Floating-Point Values

For floating-point data, EduFramework provides `Serial1_printFloat()` and `Serial1_printlnFloat()`. In the current version of EduFramework, floating-point values are displayed with three digits after the decimal point [4] .

API	Syntax	Function
`Serial1_printFloat()`	`Serial1_printFloat(value);`	Prints a floating-point value without automatically terminating the line.
`Serial1_printlnFloat()`	`Serial1_printlnFloat(value);`	Prints a floating-point value and terminates the line.

Table 3.6.2. APIs for printing floating-point values through Serial1

Example:

```
Serial1_print("Temperature: ");
Serial1_printFloat(temperature);
Serial1_println(" C");
```

These APIs will be reused in later labs related to ADC and sensors when measured values need to be observed directly on the Serial Monitor.

6.3 Extension Exercise: Counting Button Presses

Extend the main program to track the total number of times `BTN0` has been pressed since system startup. Each valid press event must increment the counter only once.
After each press, the Serial Monitor should display output in the following form:

```
BTN0 pressed. Count: 1
BTN0 pressed. Count: 2
BTN0 pressed. Count: 3
```

Use `Serial1_print()` together with an appropriate integer-printing API to create a line containing both descriptive text and the counter value.
Do not change the event-detection requirement of the main program: holding the button must not cause the counter to increase continuously.

EXPECTED RESULT

Each time `BTN0` is pressed and recognized as a new event, the counter value increases by one. The Serial Monitor displays the accumulated number of presses in the correct event sequence.

6.4 `Serial1` and `Serial2`

EduFramework provides two hardware Serial interfaces intended for different purposes [4] .

Interface	Peripheral	Intended Use
`Serial1`	`LPUART1`	Serial Monitor and debug information through the board debug interface.
`Serial2`	`LPUART2`	UART communication with external devices or modules through the corresponding UART pins.

Table 3.6.3. Roles of Serial1 and Serial2 in EduFramework

In labs that require data observation on a computer, `Serial1` is used as the preferred monitoring channel. `Serial2` is reserved for applications that need to exchange data with external UART devices. Data reception and command processing through UART will be introduced in a later lab.

7 References

[1] Jonathan Valvano and Ramesh Yerraballi, *Chapter 9: Serial Communication* , Introduction to Embedded Systems - The University of Texas at Austin.

[2] University of Florida, *Lab 5: Asynchronous Serial Communication* , EEL4744C - Electrical & Computer Engineering Department.

[3] Arduino, *Arduino Language Reference* , Arduino Documentation.

[4] EduFramework, *Hardware Serial API* , EduFramework Source Code.

[5] MaaZEDU, *MaaZEDU Development Board Guide* .